AISC Academic Technical Report

The Monkey and Banana Problem: AI Planning & Search Analysis

Formal State-Space Problem Formulation, Means-Ends Reasoning, and Comparative Evaluation of Breadth-First Search (BFS) vs. Depth-First Search (DFS)

Domain: Artificial Intelligence & Cognitive Science	**Target System:** Interactive AI Game & Planning Dashboard
Core Focus: Classical Symbolic Planning & State-Space Graphs	**Key Algorithms:** Breadth-First Search (BFS) & Depth-First Search (DFS)

1. Cognitive Science & Historical Background

The **Monkey and Banana Problem** is one of the foundational thought experiments and benchmarks in both Cognitive Science and Artificial Intelligence. It originated in 1925 when pioneering Gestalt psychologist **Wolfgang Köhler** conducted insight experiments on chimpanzees on the Island of Tenerife (published in *The Mentality of Apes*).

Prior to Köhler, behaviorist psychology (e.g., Edward Thorndike's trial-and-error learning) claimed that animals only learn by blind random exploration and reinforcement. Köhler showed that when presented with a banana hung out of reach and a movable wooden box elsewhere in the room, primates do not simply thrash about aimlessly. Instead, after a period of contemplation, they exhibit **insight**: formulating an internal plan to reach the box, push it directly beneath the bananas, climb atop it, and grasp the reward.

In modern Artificial Intelligence, this problem was formalized by John McCarthy and Allen Newell as an archetypal **means-ends analysis** planning domain: an agent must recognize that an immediate goal is blocked by an unsatisfied precondition, decomposing the task into chained intermediate sub-goals.

2. Formal State-Space Problem Formulation

In classical AI planning (Russell & Norvig, 2020), a deterministic, discrete search domain is formally defined by a 5-tuple: $\Pi = \langle \mathbf{S, S_0, G, A, T} \rangle$.

2.1 State Representation (S): Each state $s \in S$ is represented by the 5-tuple:

s = (monkeyPosition, boxPosition, bananaPosition, monkeyOnBox, hasBanana)
- *monkeyPosition* ∈ { Left, Center, Right }
- *boxPosition* ∈ { Left, Center, Right }
- *bananaPosition* ∈ { Left, Center, Right }
- *monkeyOnBox* ∈ { True, False }
- *hasBanana* ∈ { True, False }

State Space Size: The Cartesian product gives 3 × 3 × 3 × 2 × 2 = 108 theoretical states. After enforcing physical reachability invariants (e.g., the monkey can only climb on the box if *monkeyPosition* == *boxPosition*), the valid reachable state graph contains 24 to 36 states.

2.2 Initial State (S_0) and Goal Test (G):
- Standard Initial State S_0: (Left, Center, Right, False, False)
- Goal Condition G(s): s.hasBanana == True

2.3 Action Operators (A) & Transition Model (T):

Action	Parameters	Preconditions	Effects (State Transition)
Walk	target in {L, C, R}	!monkeyOnBox isAdjacent(monkeyPos, target)	monkeyPos <- target
Push Box	target in {L, C, R}	!monkeyOnBox monkeyPos == boxPos isAdjacent(boxPos, target)	boxPos <- target monkeyPos <- target
Climb Box	None	!monkeyOnBox monkeyPos == boxPos	monkeyOnBox <- True
Jump Down	None	monkeyOnBox == True	monkeyOnBox <- False
Grab Banana	None	monkeyOnBox == True boxPos == bananaPos !hasBanana	hasBanana <- True (Goal Achieved)

3. Why Search Algorithms Are Required

An intelligent autonomous agent cannot rely on hard-coded heuristics or myopic instinct. In this environment:

- **Multiple Branching Choices:** From any ground location, the monkey can walk left or right, push the box, or climb. The agent must deliberate which action progresses toward the goal.
- **Dead-End Traps:** Climbing the box when it is not situated beneath the banana yields a configuration from which the banana remains unreachable. The monkey must climb down to resume progress.
- **Infinite Loops & Cycles:** Reversible actions (e.g., walking Left then Right, or climbing Up then Down) create cycles in the state space graph. Without a formal search strategy and cycle detection, naive agents loop indefinitely.

4. In-Depth Comparative Evaluation: BFS vs. DFS

4.1 Breadth-First Search (BFS)

Mechanism: BFS explores the state space level-by-level using a First-In, First-Out (**FIFO**) queue frontier. It evaluates all nodes at depth d before evaluating any node at depth $d + 1$.

- **Optimality: Guaranteed Optimal.** Because each step incurs an identical unit cost, BFS mathematically guarantees finding the shortest action sequence (minimum number of steps to reach the banana).
- **Completeness: Complete.** If a valid path to the goal exists, BFS is guaranteed to find it.
- **Time Complexity:** $O(b^d)$, where b is the branching factor (~2 to 3) and d is the goal depth (~4 steps). In this problem, BFS examines roughly 15 to 30 states in under 2 milliseconds.
- **Space Complexity:** $O(b^d)$. All generated nodes must be stored in memory. In large-scale domains this exponential space is a disadvantage, but in the Monkey & Banana domain the memory footprint is negligible (< 200 KB).

Why BFS is the Benchmark Solver: *For college robotics and AI planning demonstrations, BFS is the ideal algorithm because it produces the cleanest, most efficient human-like plan: Walk → Push → Climb → Grab, without meandering detours.*

4.2 Depth-First Search (DFS)

Mechanism: DFS dives as deeply as possible along a single branch of the state graph using a Last-In, First-Out (**LIFO**) stack before backtracking when hitting a dead end or visited state.

- **Optimality: Not Optimal.** DFS returns the *first* goal node it encounters. Depending on successor ordering, it may choose a convoluted route (e.g., walking Left, climbing prematurely, jumping down, walking back and forth) resulting in an 8- to 12-step plan instead of the 4-step optimum.
- **Completeness: Complete only with cycle detection.** In graphs with reversible actions, DFS without a closed list will enter an infinite loop (e.g., Left ↔ Center). Our implementation enforces a visited set to guarantee termination.
- **Time Complexity:** $O(b^m)$, where m is the maximum depth bound. It can explore far deeper than necessary before discovering a simple solution.
- **Space Complexity:** $O(b \times m)$ (Linear memory). DFS only maintains the current path and unexplored siblings, making it memory-efficient in very deep trees.

5. Systematic Feature Comparison Matrix

Criterion	Breadth-First Search (BFS)	Depth-First Search (DFS)	Significance in Monkey & Banana Doma
Frontier Structure	FIFO Queue	LIFO Stack / Recursion	Dictates exploration order across the state space.
Path Optimality	Guaranteed Minimum Steps	Non-Optimal (May take detours)	BFS achieves 4 steps; DFS may take 7 to 10 steps.
Completeness	Always Complete	Complete with cycle check	Cycle checking prevents infinite ping-pong between tiles.
Time Complexity	O(b^d) where d = goal depth	O(b^m) where m = max depth	BFS finds shallow goal faster in this specific topology.
Space Complexity	O(b^d) (Exponential)	O(b * m) (Linear)	Not a constraint here due to compact state size (< 100 stat
Search Tree Shape	Broad, balanced, level-by-level	Deep, elongated asymmetric spine	Visualized clearly in the app's Search Tree panel.

6. Analysis of the Four Named Possibilities (A, B, C, D)

To rigorously test search behaviors, the application implements four distinct canonical initial configurations (Possibilities A, B, C, and D). In each scenario, BFS consistently finds the mathematically optimal shortest path, whereas DFS exhibits a depth detour (prematurely climbing the box before moving it under the banana):

Case	Initial Positions (Monkey, Box, Banana)	BFS Optimal Steps	DFS Detour Steps	Observable Difference & Detour Ratio
Possibility A	Monkey: Left Box: Center Banana: Right	4 Steps (Walk, Push, Climb, Grab)	6 Steps (Walk, Climb, Jump Down, Push, Climb, Grab)	DFS prematurely climbs at Center before pushing the box
Possibility B	Monkey: Center Box: Left Banana: Right	5 Steps (Walk, Push, Push, Climb, Grab)	7 Steps (Walk, Climb, Jump Down, Push, Push, Climb, Grab)	Box must traverse two room segments; DFS climbs at Le
Possibility C	Monkey: Right Box: Right Banana: Left	4 Steps (Push, Push, Climb, Grab)	6 Steps (Climb, Jump Down, Push, Push, Climb, Grab)	Monkey begins already at the box; DFS immediately climb
Possibility D	Monkey: Center Box: Right Banana: Center	4 Steps (Walk, Push, Climb, Grab)	6 Steps (Walk, Climb, Jump Down, Push, Climb, Grab)	Banana in Center; DFS climbs at Right first, discovering b

7. Summary & Educational Conclusion

By modeling the Monkey and Banana problem with formal state predicates and evaluating it via both Breadth-First Search and Depth-First Search, students gain immediate insight into the trade-offs of classical AI. BFS guarantees the optimal solution path, providing the benchmark for means-ends rationality. Meanwhile, DFS reveals the risks of cycle loops and redundant steps in unguided exploration. The integrated web application brings these theoretical principles to life through synchronized visual animation, live search trees, and speech feedback.